AUTONATEAI SCREEPS LEAGUE

Tutorial 03: Roles

A practical local-server Screeps module **docs/tutorials/03-roles.md**

Episode 3: Job Descriptions

"Every creep in this colony does the same job," your collaborator says. "Harvest, walk, deliver, repeat. That's a two-person startup where everyone's Head of Everything. Cute for a minute. Falls apart the second you need a third hire to actually specialize."

You need energy going somewhere other than the spawn now. The controller's been sitting there since the day you landed, ignored the way founders ignore paperwork they know they'll need eventually.

Goal

Refactor the single `runHarvester` function from Episode 2 into two distinct roles, each in its own file, dispatched by a population target instead of a hardcoded name list:

- `role.harvester.js` – the behavior you already built
- `role.upgrader.js` – a new behavior that feeds the room controller
- `main.js` – spawns creeps by role count and dispatches by `creep.memory.role`

Prerequisites

Tutorial 02 is complete:

- Two harvesters exist, each with a `sourceId` assigned in Memory.
- `getAssignedSource` and `runHarvester` are working in `main`.

Step 1: Understand Why One File Won't Scale

Open the in-game script editor. The Screeps runtime lets you create more than one file per branch – each becomes a `require`-able module, the same way Node.js modules work.

This matters now because the next role you add – upgrader – needs its own harvest logic, its own delivery logic, and its own target. Cramming a second `if (creep.memory.role === 'upgrader')` branch into `main` works for one role. It stops being readable at three.

Step 2: Create `role.harvester.js`

Add a new file named `role.harvester` (the editor will show it as `role.harvester.js`). Move the harvester logic from Episode 2 into it, exported as a module:

```
const roleHarvester = {
  run(creep) {
    if (creep.store.getFreeCapacity() > 0) {
      const source = this.getAssignedSource(creep);
      if (creep.harvest(source) === ERR_NOT_IN_RANGE) {
        creep.moveTo(source, { visualizePathStyle: { stroke: '#ffaa00' } });
      }
      return;
    }

    const spawn = Game.spawns.Spawn1;
    if (creep.transfer(spawn, RESOURCE_ENERGY) === ERR_NOT_IN_RANGE) {
      creep.moveTo(spawn, { visualizePathStyle: { stroke: '#ffffff' } });
```

```
    }
  },

  getAssignedSource(creep) {
    if (!creep.memory.sourceId) {
      const sources = creep.room.find(FIND_SOURCES);
      const claimCounts = {};
      sources.forEach((source) => {
        claimCounts[source.id] = 0;
      });

      for (const name in Game.creeps) {
        const assignedId = Game.creeps[name].memory.sourceId;
        if (assignedId && assignedId in claimCounts) {
          claimCounts[assignedId] += 1;
        }
      }

      sources.sort((a, b) => claimCounts[a.id] - claimCounts[b.id]);
      creep.memory.sourceId = sources[0].id;
    }

    return Game.getObjectById(creep.memory.sourceId);
  },
};

module.exports = roleHarvester;
```

Step 3: Create `role.upgrader.js`

Add a new file named `role.upgrader`:

```
const roleUpgrader = {
  run(creep) {
    if (creep.memory.upgrading && creep.store[RESOURCE_ENERGY] === 0) {
      creep.memory.upgrading = false;
    }
    if (!creep.memory.upgrading && creep.store.getFreeCapacity() === 0) {
      creep.memory.upgrading = true;
    }

    if (creep.memory.upgrading) {
      const controller = creep.room.controller;
      if (creep.upgradeController(controller) === ERR_NOT_IN_RANGE) {
        creep.moveTo(controller, { visualizePathStyle: { stroke: '#ffffff' } });
      }
      return;
    }

    const sources = creep.room.find(FIND_SOURCES);
    if (creep.harvest(sources[0]) === ERR_NOT_IN_RANGE) {
      creep.moveTo(sources[0], { visualizePathStyle: { stroke: '#ffaa00' } });
    }
  },
};

module.exports = roleUpgrader;
```

Notice the upgrader does not use `getAssignedSource`. It doesn't need to split load across sources yet — one upgrader pulling from whichever source is closest is fine. That's a deliberate scope cut, not an oversight: don't add the harvester's load-balancing to a role that doesn't have the same problem yet.

Step 4: Rewrite `main` to Dispatch by Role

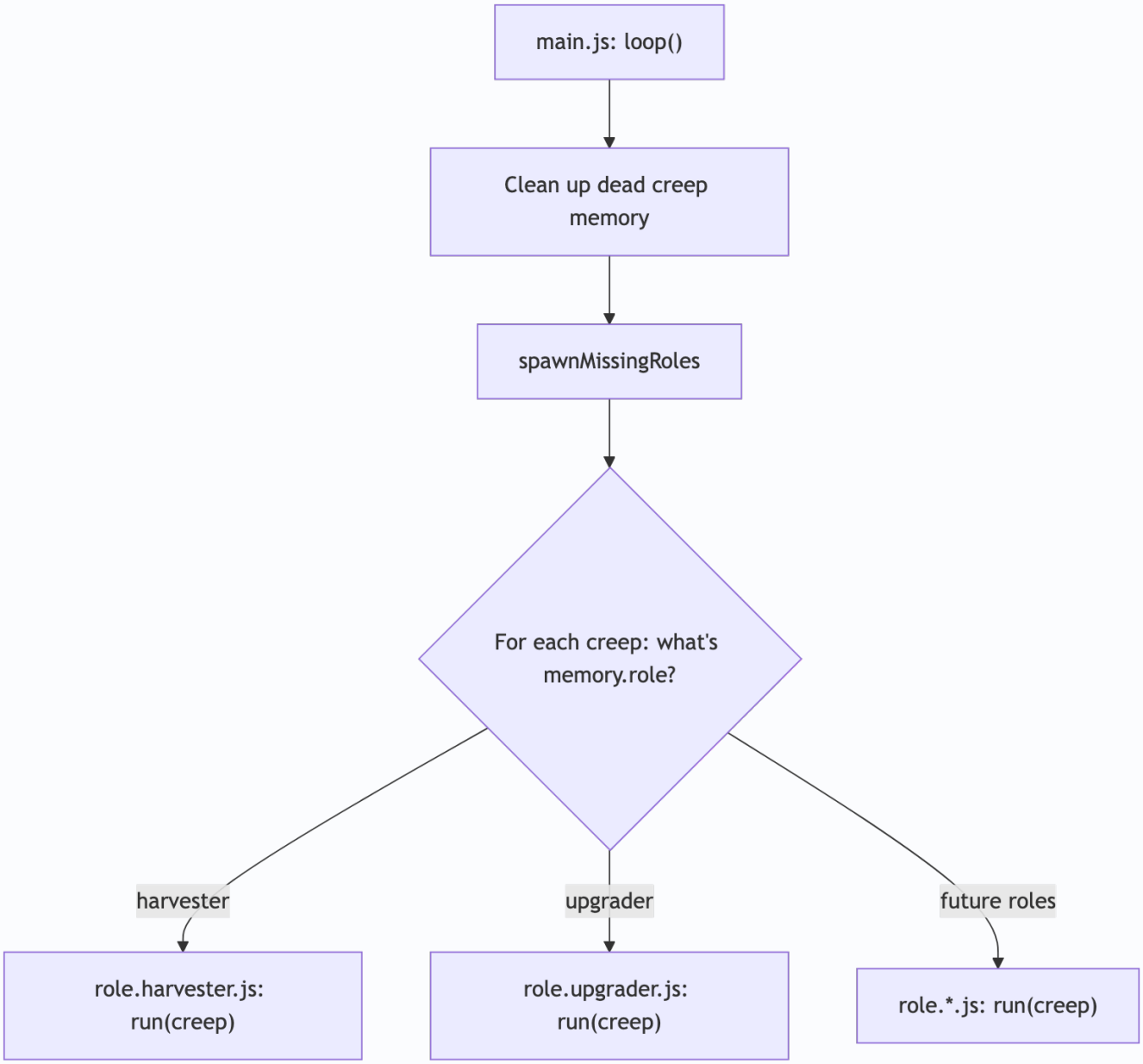
Replace `main` with:

```
const roleHarvester = require('role.harvester');
const roleUpgrader = require('role.upgrader');

const POPULATION = {
  harvester: 2,
  upgrader: 1,
};

module.exports.loop = function () {
  for (const name in Memory.creeps) {
    if (!Game.creeps[name]) {
      delete Memory.creeps[name];
    }
  }

  spawnMissingRoles();

  for (const name in Game.creeps) {
    const creep = Game.creeps[name];
    if (creep.memory.role === 'harvester') {
      roleHarvester.run(creep);
    } else if (creep.memory.role === 'upgrader') {
      roleUpgrader.run(creep);
    }
  }
};

function spawnMissingRoles() {
  const spawn = Game.spawns.Spawn1;
  if (spawn.spawning) {
    return;
  }

  for (const role in POPULATION) {
    const currentCount = _.filter(Game.creeps, (creep) => creep.memory.role === role).length;
    if (currentCount < POPULATION[role]) {
      const name = `${role}${Game.time}`;
      spawn.spawnCreep([WORK, CARRY, MOVE], name, { memory: { role } });
      break;
    }
  }
}
```

Save all three files.

`_` is lodash, available globally in the Screeps runtime — you don't need to `require` it.

This is the architecture every future episode builds on — `main.js` never contains behavior itself, only dispatch:



Adding a role later means: write a new `role.*.js` file, add one line to `ROLE_BODIES` / `POPULATION`, add one `else if` branch. `main.js` itself barely changes across the rest of this season.

Checkpoint:

- Existing creeps (`Harvester1`, `Harvester2`) stop moving. They have no `memory.role`, so neither `if` branch matches them.
- This is expected. Continue to Step 5.

Step 5: Retire the Old Creeps

The creeps from Episodes 1–2 were spawned without a `role` in memory, so the new dispatch logic ignores them. Clear them out and let the population system take over:

```
Game.creeps.Harvester1.suicide();
Game.creeps.Harvester2.suicide();
```

Checkpoint:

- Within a few ticks, `spawnMissingRoles` notices the room has 0 harvesters and 0 upgraders and starts spawning to match `POPULATION`.
- New creeps are named things like `harvester12345` and `upgrader12349` (role plus spawn tick).
- Each new creep has `creep.memory.role` set correctly:

```
_.map(Game.creeps, (creep) => [creep.name, creep.memory.role])
```

Step 6: Watch the Controller Move

Open the room in the client and look at the controller's progress bar. It should start ticking upward once the upgrader is delivering energy.
Checkpoint:

```
Game.spawns.Spawn1.room.controller.progress
Game.spawns.Spawn1.room.controller.progressTotal
```

`progress` should increase over time. It won't reach `progressTotal` in this episode — that's Episode 4.

> 📸 **Screenshot placeholder:** The room controller's progress bar in the client, visibly filling — the first moment upgrading stops being an abstract number and becomes something you can watch move.

Troubleshooting

If no creeps spawn at all:

```
Game.spawns.Spawn1.spawning
Game.spawns.Spawn1.store[RESOURCE_ENERGY]
```

If a creep exists but does nothing:

```
Game.creeps['<name>'].memory
```

Confirm `role` is set to exactly `'harvester'` or `'upgrader'` — a typo here means neither branch in `main` matches. If `require('role.harvester')` throws, confirm the file is named exactly `role.harvester` in the editor (no `.js` in the `require` call) and that `module.exports` is set at the bottom of the file.

Completion Criteria

You are done when:

- `role.harvester.js`, `role.upgrader.js`, and `main.js` exist as separate files.
- The population system spawns 2 harvesters and 1 upgrader without any hardcoded names.
- The controller's `progress` value is increasing.
- Killing any creep results in a same-role replacement, not a name-specific one.

Learning Notes

After completing the tutorial, write down:

- What broke when you suicided the old creeps, and why?
- What would happen if `POPULATION` had a role with a count of `0`? Does `spawnMissingRoles` handle that correctly?
- Why does the upgrader use a `memory.upgrading` flag instead of just checking `getFreeCapacity()` every tick the way the harvester does?
- If you added a third role right now, what's the minimum set of changes across all three files?

Next: Episode 4 — The Controller's Price

The controller is moving. It isn't moving fast, and it isn't moving toward anything yet.

"Every level that controller climbs unlocks something," your collaborator says. "Extensions, bigger creeps, more energy capacity. Right now you're upgrading it because you can. Next episode, you're upgrading it because you actually need what's on the other side — the difference between a vanity metric and a real one."

See `docs/roadmap.md` for the full season.